STAR-SRAM: 16-bit Floating-Point SRAM-Based Digital Computing-in-Memory Macro in a 28 nm

Chuan-Tung Lin, *Student Member, IEEE*, Jonghyun Oh, *Member, IEEE*, and Mingoo Seok, *Senior Member, IEEE*

***Abstract*—A digital computing-in-memory (DCIM) macro emerges as a promising building block in a deep neural network (DNN) accelerator. To better support DNN workloads, circuit designers aim to improve three main metrics for macros: energy efficiency, compute density, and weight density. Improvements in those metrics directly translate into reduced energy consumption, shorter inference latency, and a smaller silicon footprint at the accelerator level. This article introduces STAR-SRAM, a DCIM macro designed for 16-bit brain floating-point (BF16) arithmetic. The macro is designed using a top-down approach. We started the design process with model-based parameter optimization, in which we determined the optimal design parameters that maximize the figure of merit (FoM) of a DCIM macro. Then, we developed several circuit techniques that further improve the FoM, including pre-multiply symmetric alignment, approximate multiplier circuits, and sparsity-aware wordline, clock, and input gating techniques. Fabricated in 28-nm CMOS, STAR-SRAM achieves the state-of-the-art FoM with up to 43.06 TFLOPS/W, 1.89 TFLOPS/mm^2, and 400 Kb/mm^2.**

***Index Terms*—Compute-in-memory, deep learning, digital circuits, in-memory computing, neural network accelerator, SRAM.**

I. Introduction

THE deep neural networks (DNNs) have demonstrated very high performance across numerous applications [1], [2], [3], [4], [5], which has motivated circuit designers to propose a large number of SRAM-based computing-in-memory (CIM) macros in recent years. Such a macro embeds computation hardware directly within a memory macro to execute multiply-and-accumulate (MAC) operations [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16]. A CIM macro can simultaneously activate multiple SRAM rows for parallel MAC operations, avoiding the overhead of row-by-row SRAM access and thereby significantly enhancing both energy efficiency and computational throughput.

Recently, significant research attention has been given to the digital version of the CIM (DCIM) macro [17], [18], [19], [20], [21], [22], [23], as it can support high-precision arithmetic, such as floating-point arithmetic. Supporting high-precision arithmetic in the DNN workload remains critical, even though many DNN models adopt low-precision arithmetic and achieve competitive accuracy, because it enables higher accuracy and simplifies model development.

In those DCIM works, circuit designers often aim to improve three metrics: energy efficiency, compute density, and weight density. Improvements in those metrics directly translate to reduced energy consumption, shortened inference latency, and smaller silicon footprint at the accelerator level [17], [24], [25], [26], [27], [28], [29], [30]. Specifically, defined as TFLOPS/W, the higher energy efficiency reduces the energy consumption of an accelerator for performing a given inference workload. Also, measured in TFLOPS/mm^2, higher compute density reduces the latency of an accelerator for a given silicon area. Last but not least, the higher weight density, defined as Kb/mm^2, reduces silicon area and cost for a given memory capacity requirement of an accelerator [24], [25]. It affects the amount of weight data the accelerator can buffer and has multiple implications for both on- and off-accelerator data movement.

Existing floating-point CIMs generally exhibit low energy efficiency, compute density, and weight density, especially compared to their fixed-point counterparts, because floating-point arithmetic involves complex operations such as comparisons and bit shifts. To address those challenges, recent works have proposed the use of approximate arithmetic (e.g., block floating-point [31]) and analog-mixed-signal (AMS) circuits [29]. Nonetheless, these techniques increase the error in the computation results. For example, the AMS circuits can increase the root-mean-square error (RMSE) across process, voltage, and temperature (PVT) variations.

In light of this, we propose novel DCIM macro circuits, called STAR-SRAM, that support 16-bit floating-point arithmetic while simultaneously balancing all three metrics. We adopt a top-down approach in designing STAR-SRAM. We start our design process with model-based design parameter exploration. Specifically, we develop a range of models that predict the three metrics of a DCIM macro as a function of various design parameters. Then, we sweep those parameters to find the optimal design point. The model-based design parameter exploration enables us to make three design decisions: bit-serial versus bit-parallel input, optimal hardware reuse, and approximation level.

Furthermore, we develop several circuit techniques to reduce the arithmetic hardware size and power consumption. Specifically, we propose a pre-multiply symmetric alignment

Received 26 September 2025; revised 23 February 2026 and 21 April 2026; accepted 16 May 2026. Date of publication 2 June 2026; date of current version 27 August 2026. This article was approved by Associate Editor Chia-Hsiang Yang. This work was supported in part by NSF under Grant IIP-1919147 and in part by COGNISENSE, one of the Seven Centers in JUMP 2.0, a Semiconductor Research Corporation (SRC) Program Sponsored by Defense Advanced Research Projects Agency (DARPA). *(Corresponding author: Mingoo Seok.)*

Chuan-Tung Lin was with the Department of Electrical Engineering, Columbia University, New York, NY 10027 USA. He is now with Google, Madison, WI 53703 USA.

Jonghyun Oh was with the Department of Electrical Engineering, Columbia University, New York, NY 10027 USA. He is now with Apple, Cupertino, CA 95014 USA.

Mingoo Seok is with the Department of Electrical Engineering, Columbia University, New York, NY 10027 USA (e-mail: ms4415@columbia.edu).

Digital Object Identifier 10.1109/JSSC.2026.3694927

0018-9200 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:21:40 UTC from IEEE Xplore. Restrictions apply.

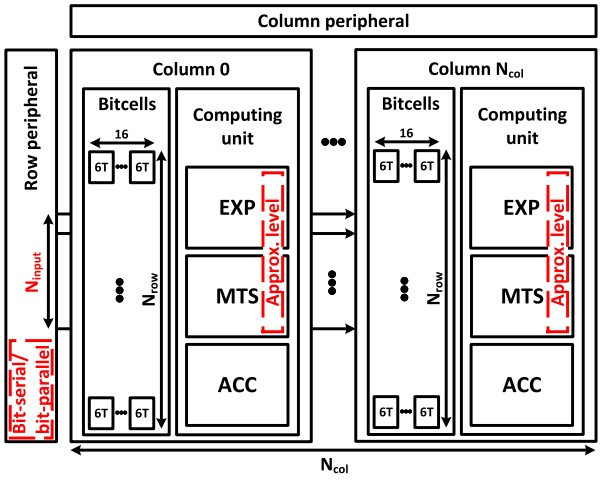


Fig. 1. Generic CIM macro architecture we consider in the model-based design parameter exploration. Through the exploration, we have made the three key decisions, namely bit-serial versus bit-parallel input, the degree of hardware reuse, and the approximation level.

scheme, where the decimal points of the inputs and weights are aligned *before* multiplication. The conventional floating-point MAC algorithm aligns the decimal points after multiplication (i.e., before accumulation). We also develop an approximate multiplier that is co-optimized with the proposed alignment scheme. Together, they strike a balance between arithmetic hardware area savings and RMSE penalty. Also, we propose sparsity-aware wordline, clock, and input-gating techniques to further improve energy efficiency. Last but not least, we update the macro circuits to better support a depthwise (DW) convolution layer, yet without compromising area and energy efficiency for standard convolution computation.

We prototyped a STAR-SRAM test chip in a 28-nm process. The measurement results show that STAR-SRAM achieves up to 43.06 TFLOPS/W, up to 1.89 TFLOPS/mm^2, and 400 Kb/mm^2 in 16-bit brain floating-point (BF16) arithmetic, significantly outperforming prior floating-point DCIM macros in figure of merit (FoM). The remainder of this article is as follows. Section II presents the modeling and design parameter exploration. In Section III, we present the proposed STAR-SRAM architecture and circuits. In Section IV, we show the chip prototype and measurement results. Finally, Section V concludes this article.

II. Model-Based Design Parameter Optimization

A. Key Design Choices

In this work, we pursue a model-based design parameter optimization to identify optimal design choices. Fig. 1 shows a generic CIM macro architecture that we consider for the exploration. Without loss of generality, we aim to support BF16. It has a 1-bit sign, an 8-bit exponent, and a 7-bit mantissa. The CIM macro contains N_{col} columns, a row peripheral, and a column peripheral. Each column includes $N_{row} \times 16$ 6T bitcells and a computing unit. The computing unit consists of one exponent (EXP) unit, one mantissa (MTS) unit, and one accumulation (ACC) unit.

The macro performs vector matrix multiplication. Typically, the input is a vector, and the weight is a matrix. We store the input vector in the row peripheral and the weight matrix in the bitcells. The EXP unit takes one or multiple N_{input} pairs of input and weight values and generates bit-shift amounts to align the mantissas of the inputs and weights. On the other hand, the MTS unit aligns the mantissa and performs MAC operations. Finally, the ACC unit accumulates the results across all pairs of inputs and weights.

We explore three major design parameters. First, we decide on the input scheme mainly between bit-serial and bit-parallel. We decided to consider these two cases because they are the two most representative and popular configurations. Second, we determine the optimal degree of hardware reuse. The degree of hardware reuse is related to the number of input elements we feed per cycle (N_{input}), which equals the number of multipliers in each column. Third, we determine the level of approximation by comparatively evaluating exact arithmetic, block floating-point [31], and other methods. We focus on those parameters because we believe they have the greatest impact on macro's performance and efficiency. Indeed, we cannot explore all other possible design parameters, as this would require a prohibitive amount of effort.

Most existing DCIM *circuit-level* works focus on a single design point and skip design-space exploration. But several non-prototype works recently have investigated the design-space analysis of CIM designs. For example, Diao et al. [22] proposed an automated compiler framework that explores digital CIM design configurations with multiple-precision support, primarily focusing on design automation and the rapid generation of CIM macros. Similarly, Zhang et al. [32] presented an end-to-end automated flow for analog CIM. Sun et al. [33] performed a cross-domain benchmarking study comparing analog and digital CIM using quantitative modeling. Spetalnick and Raychowdhury [34] provided an analog SRAM-CIM design space analysis focusing on key architectural parameters and their impact on efficiency. However, those works keep their studies mostly at the model level, offering no circuit-level techniques or a silicon prototype. They also spend a significant amount of effort on an analog CIM. Also, prior work focuses on analysis rather than proposing optimal design choices. In contrast, this work performs design-space exploration alongside developing new circuit-level techniques, proposing an optimal combination of multiple design choices. These investigations result in a silicon prototype. Also, this work focuses exclusively on the digital floating-point CIM.

B. Bit-Serial Versus Bit-Parallel Input

We must decide how to feed an input vector to the CIM macro. The bit-serial scheme is commonly used in fixed-point CIM macros [18], [26], [35], [36], [37], [38]. It feeds each bit of all or some elements in a vector every cycle. This scheme allows the macro to employ potentially simple bit-serial arithmetic hardware, such as a 1-bit multiplier, yet at the cost of reduced computing throughput. On the other hand, the bit-parallel scheme supplies all bits of all or some elements in a vector in a single cycle. It greatly improves computing

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:21:40 UTC from IEEE Xplore. Restrictions apply.

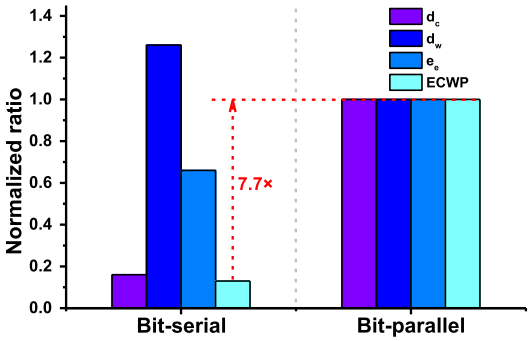


Fig. 2. Design parameter exploration results for bit-serial versus bit-parallel input.

throughput. However, it requires larger arithmetic hardware than the bit-serial counterpart.

To make this design decision, we perform a model-based investigation. We compare two most important design points: 1) fully bit-parallel and 2) bit-serial for mantissa and bit-parallel for exponents (hereafter we call it bit-serial for simplicity). We do not consider the bit-serial exponent design. This is because we perform mainly additions with exponents, and the bit-serial adder provides limited hardware savings for low-precision arithmetic hardware [39].

To compare those two design points, we have developed models of energy efficiency (e_e), compute density (d_c), and weight density (d_w) of the macro. We also define an FoM called ECWP, which is the product of the three metrics above. We include the model derivation details in Appendix A for interested readers. Fig. 2 shows the results of the comparisons.[1] The bit-serial input achieves higher weight density but lower compute density and energy efficiency than the bit-parallel one. As a result, the bit-parallel input achieves 7.7× higher ECWP than the bit-serial counterpart. Based on the results, we decided to employ the bit-parallel input scheme for our macro.

C. Optimal Degree of Hardware Reuse

We now explore the optimal degree of hardware reuse. An extreme design option is to have no hardware reuse. In other words, we employ sufficient arithmetic hardware to support the parallel computation of all pairs of inputs and weights in a column simultaneously in a single cycle. This architecture would achieve very high computing throughput but also consume a very large amount of arithmetic hardware. The other extreme design choice is to use arithmetic hardware to compute only one pair of input and weight elements. Such maximum hardware reuse requires the smallest arithmetic hardware, but can achieve only limited computation throughput.

To determine the optimal degree of hardware reuse, we also pursue a similar model-based exploration approach. Again, we develop the simple closed-form formula for those metrics (d_w, d_c, e_e) and FoM (ECWP) as a function of N_{input}. We include the model derivation details in Section V-A for interested readers. We then evaluate the formula by sweeping N_{input} from 1 to 64 for N_{row} of 64. This sweep implies that we employ N_{input} multipliers in a column and N_{row}/N_{input} (= 1 to 64) pairs of inputs and weights share the same arithmetic hardware. Fig. 3 shows the results of the exploration, where N_{input} of 4 (i.e., having four multipliers in a column and sharing the arithmetic hardware across 16 pairs of inputs and weights) gives the best ECWP metric, which is 8.3X better than the worst design choice (N_{input} = 64).

[1]Note that in Figs. 2–5, we intentionally present the data in normalized form because they are derived from analytical models. Reporting absolute values would require us to determine several hyperparameters for the models, which would entail implementation details and can therefore be done only after we implement circuits.

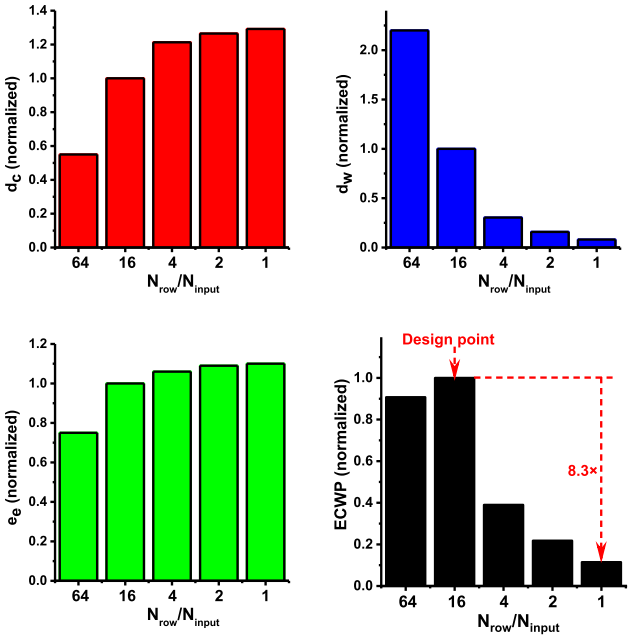


Fig. 3. Design parameter optimization results for the degree of hardware reuse. N_{row}/N_{input} of 16 is found to be optimal. N_{row}/N_{input} refers to the number of weights that share one hardware multiplier. We assume the bit-parallel input.

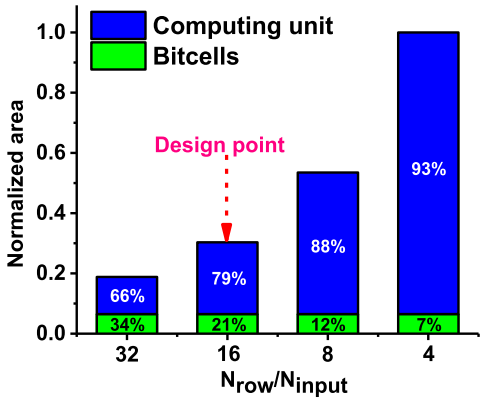


Fig. 4. Area and breakdown across different levels of hardware reuse. N_{row}/N_{input} of 16 is our design point.

D. Approximation Level

Before optimizing the approximation level, we evaluated the CIM macro's area and its breakdown using the models

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:21:40 UTC from IEEE Xplore. Restrictions apply.

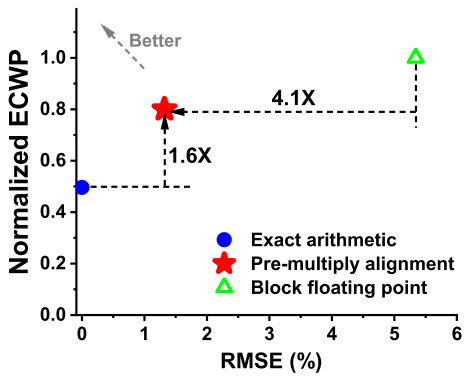


Fig. 5. Design parameter exploration results for the approximation level.

developed in Sections II-B and II-C. Fig. 4 shows the results. Setting N_{input} to 16 yields a smaller area, especially compared to N_{input} = 4. However, even at N_{input} = 16, the computing hardware still takes 79% of the total area.

To reduce such arithmetic hardware area overhead, recent work has proposed adopting approximate arithmetic, such as block floating-point arithmetic [31]. In this work, we explore other approximation schemes, including our own proposal, and evaluate them using the ECWP and RMSE. We have considered three approximation schemes: exact arithmetic, block floating-point, and pre-multiply alignment. The latter is what we devised and is different from [29]. It aligns the mantissas before multiplication, while the prior works align before every two-input addition. We will describe the scheme in detail in Appendix B.

We then develop the ECWP and RMSE models for each scheme. Appendix B will detail the model development process for the interested readers. Fig. 5 shows the results of the exploration. The exact arithmetic indeed achieves zero RMSE, but exhibits 38% worse ECWP than the pre-multiply alignment. On the other hand, the block floating point achieves a 25% higher ECWP than pre-multiply alignment. However, it causes the largest RMSE error of 5.34%. Based on the result, we choose to employ the pre-multiply alignment in our design.

III. Proposed STAR-SRAM CIM Macro

A. STAR-SRAM Architecture

Fig. 6 left shows the STAR-SRAM's architecture. The macro has 16 columns (Column 0–15). Each column consists of four submodules: 1) four bitcell groups for exponents (bitcell_group_exp[3:0]); 2) four bitcell groups for sign and mantissa (bitcell_group_man[3:0]); 3) exponent computing module (EXP); and 4) mantissa computing module (MTS). The EXP and MTS modules are cascaded and pipelined in two stages. They contain the following submodules: 1–8-bit shifters, 8-bit comparators, 8-bit adders, and 8-bit multipliers (Fig. 6 right). The 8-bit multiplier produces a 16-bit product, which is then padded with the sign bit and the leading 1 bit to form an 18-bit signed product.

Each bitcell group has 16 × 8 6T bitcells. The 16 bitcells in a column adopt the split wordline scheme, sharing two local bitlines (LBL and LBLb) which drive a 1-bit multiplier and two global bitlines (GBL and GBLb). We employ a single-ended read access scheme [17], [26], which helps reduce the switching power consumption associated with LBL precharging. The short LBLs, which are connected to only 16 bitcells, mitigate the risk of read-upset. Monte Carlo simulations confirm that the read-upset margin remains positive under the 12-σ worst case scenario of transistor mismatches.

The static single-ended readout scheme improves energy efficiency at a small area cost. Having two wordlines increases the macro area by only 4%. The overhead is small because the drivers take only a small portion of the macro area. On the other hand, the static readout reduces the macro's energy consumption by more than 2× compared to the conventional dynamic readout, as it does not perform a costly bitline precharge operation.

B. Pre-Multiply Symmetric Alignment

We propose the pre-multiply symmetric alignment scheme, which aligns the mantissas before multiplication and performs the multiplication *symmetrically* between the input and the weight. Fig. 7(a) shows the hardware for such a pre-multiply symmetric alignment scheme, which aligns eight mantissae (of four inputs and four weights) before multiplication. The EXP module calculates the shift amounts, and the MTS module then shifts the input and weight mantissas by up to 8 bits. Note that we shift the weight data on the fly and feed the shifted values to the multipliers. Therefore, the weight data remains stationary and unchanged in the 6 T bitcells.

Fig. 7(b) shows an example. The EXP module starts the process by finding the maximum exponent sum. In this example, the IN_{e0}- W_{e0} pair has the largest exponent sum (8'b 00010011 + 8'b00010011 = 8'b00100110). Then, it calculates the shift amount for each input-weight pair by subtracting the maximum exponent sum from each exponent sum. For example, $IN_{e0} + W_{e0} - IN_{e2} - W_{e2}$ = 8'b2. Thus, we can set the total shift amount for this pair to 2 bits. We divide it equally/symmetrically between the two mantissae, so the MTS module shifts IN_{m2} and W_{m2} by 1 bit. Note that the total shift amount in this example is even, thus we symmetrically split it between the two mantissae. However, if it is an odd number, it shifts the input by one additional bit.

The proposed scheme reduces hardware requirements. Conventional exact floating-point arithmetic aligns two mantissae before every two-input addition. Such pre-accumulation alignment requires substantial arithmetic hardware. Fig. 7(c) shows the hardware for such an exact MAC operation for four pairs of inputs and weights. It requires two 1–16-bit shifters, one 1–17-bit shifter, and three 8-bit comparators. On the other hand, the proposed pre-multiply needs eight 1–8-bit shifters and three 8-bit comparators. Compared with pre-accumulation alignment, the proposed scheme achieves 19% area savings in the arithmetic hardware at a small increase in RMSE of 0.974%. Wu et al. [29] employed a similar pre-multiply alignment scheme, but it can align only input mantissae by up to 8 bits. However, in the worst case, we need to shift the input mantissas by 16 bits. If so, it truncates too many bits, resulting in a large error.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:21:40 UTC from IEEE Xplore. Restrictions apply.

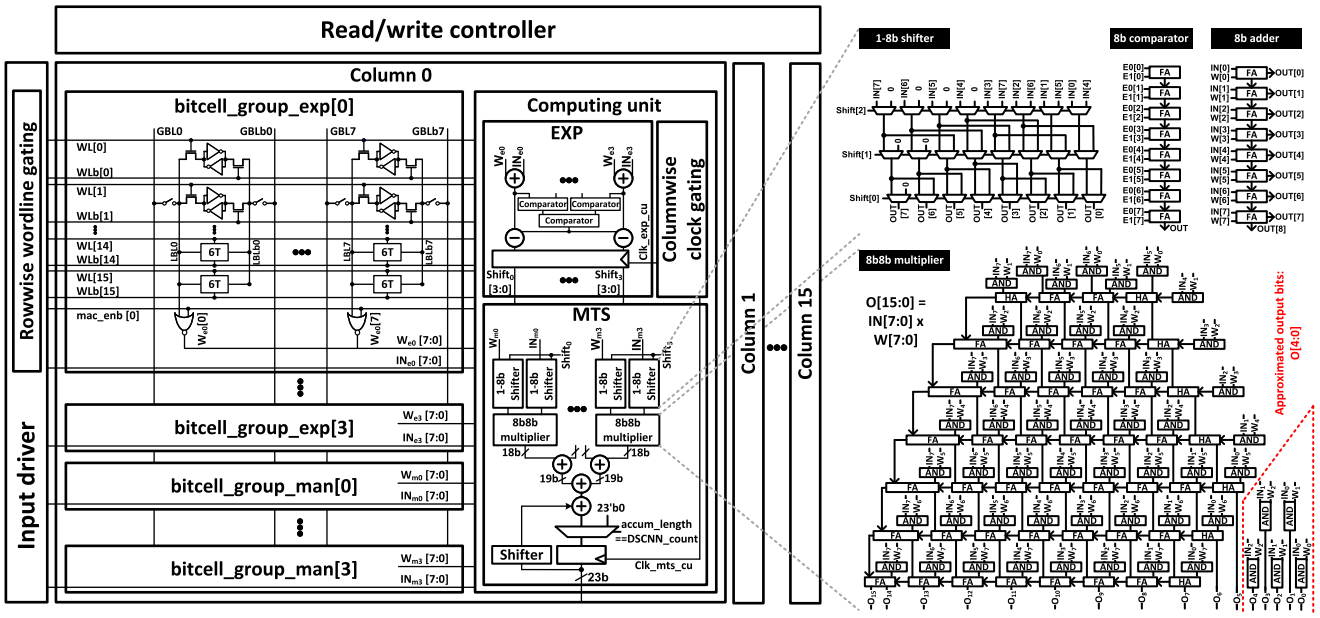


Fig. 6. STAR-SRAM architecture.

C. Approximate Multiplier

To further reduce the footprint of arithmetic hardware, we design an approximate multiplier that produces the product of an input and a weight mantissa. Fig. 8 shows the example of the partial products it intermediately generates. The 8-bit mantissa multiplication produces a total of 64 1-bit partial products (pp[7:0][7:0]). The multiplier accumulates the partial products and produces one 16-bit product ($O_{15} \ldots O_0$). Note that in this illustration, we ignore the sign bit and the leading 1 bit; they are, however, added later to form the 18-bit product before accumulation.

We aim to truncate some of the partial products while minimizing their impact on RMSE. One naive approach is to remove some of the least significant bits (LSBs) of the product. However, we find that it yields a significant error. Instead, we identify and preserve the important partial products (in red) and discard ten irrelevant partial products (those X-ed). The partial products in red are important because the MSB of the product depends on them. Recall that the proposed symmetric alignment scheme shifts the input mantissae and weight mantissae equally if the number of shifts is even. If the number of shifts is odd, it shifts the input mantissae by one more bit. This setup ensures that the MSBs (i.e., the leading 1s) of the product always depend on $IN_X W_X$ or $IN_X W_{X+1}$, where $X \in [0, 7]$. Therefore, we should keep those partial products. Fig. 6, right bottom, shows the schematics of the final approximate multiplier.

Note that the truncation of partial products may consistently incur a negative *magnitude* error in each product. However, it does not lead to large error accumulation, since the final accumulation in a neural network workload involves products with varying signs, resulting in both positive and negative errors. These positive and negative errors tend to cancel each other out during accumulation.

We evaluate the proposed pre-multiply symmetric alignment and approximate multiplier scheme in terms of area savings and RMSE. The baseline is non-approximate, that is, pre-accumulation alignment and exact multiplier. In the analysis, we also include several approximate multipliers with different numbers of partial products eliminated. Fig. 9 shows the results, where the proposed approach achieves a 25% reduction in arithmetic hardware area at a reasonable increase of RMSE of 1.32%. It highlights co-optimizing the alignment scheme and multiplication approximation, which helps strike a balance between accuracy and area.

D. Wordline, Clock, and Data Gating

To further reduce the arithmetic hardware's power consumption, we propose the row-wise WL-gating technique. Specifically, we gate the WL if its input exponent is zero (i.e., $IN_{eX} == 0$, where X is the row index). This is because the value of a floating-point number is extremely small if its exponent value is zero. It skips SRAM access and subsequent multiplication, significantly reducing power consumption. Fig. 10 shows the implementation of the row-wise WL-gating technique, which is highlighted in blue.

Similarly, we add column-wise clock gating to the EXP and MTS units. It performs a zero-check for each input–weight pair. Specifically, it checks if either input or weight exponent is zero across four pairs of inputs and weights and accordingly asserts the zero_check[3:0] signals. If every pair has at least one zero, that is, all of zero_check[3:0] signals are high, it gates the two clocks (clk_exp_cu and clk_mts_cu) that go to the pipeline registers. Fig. 10 shows the implementation of the column-wise clock-gating technique, highlighted in red.

To further reduce power consumption, we also propose the data-gating technique. It gates the inputs to the comparators (i.e., the max exponent sum finder) and the adders in the EXP

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:21:40 UTC from IEEE Xplore. Restrictions apply.

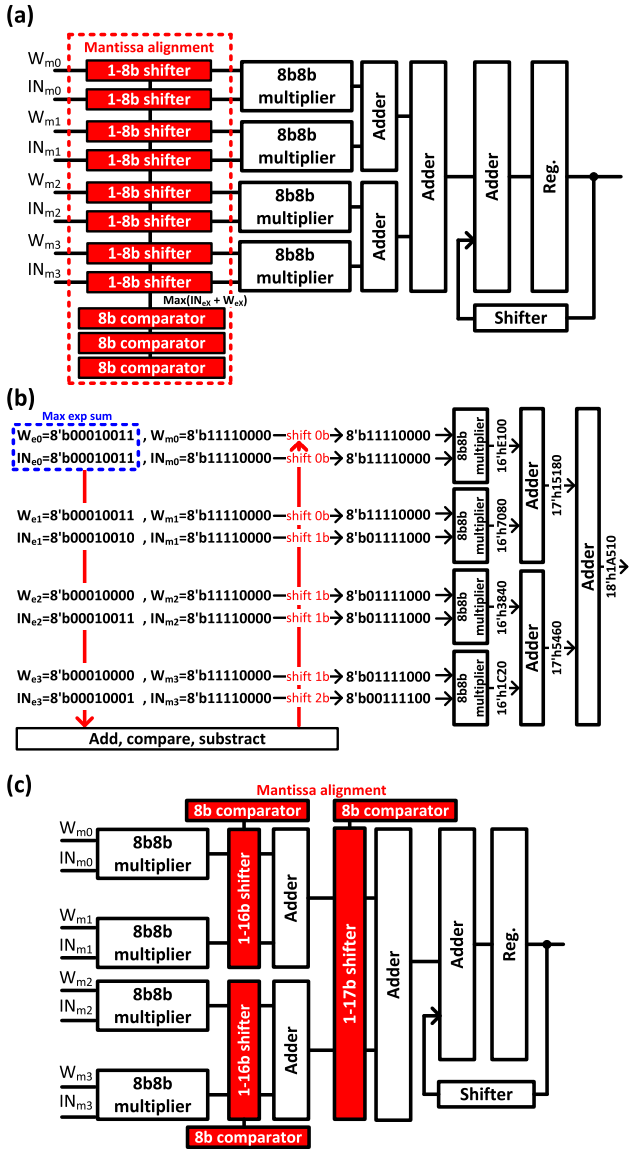


Fig. 7. (a) Block diagram for the proposed pre-multiply symmetric alignment. (b) Example of MAC operations using the pre-multiply alignment scheme. (c) Block diagram for the conventional pre-accumulation alignment.

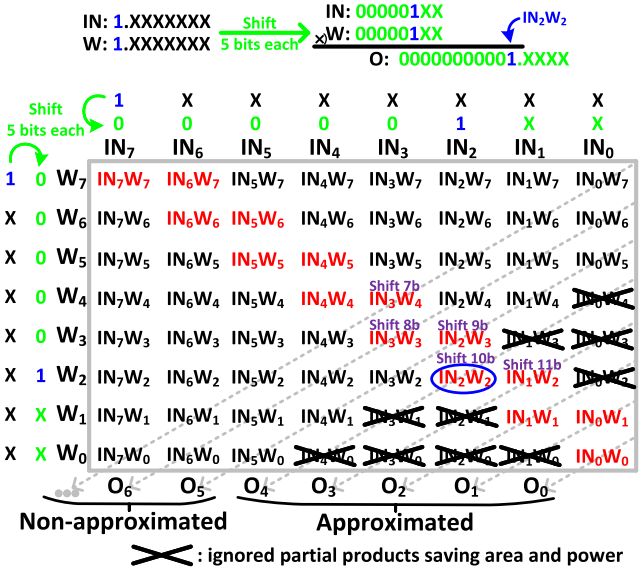


Fig. 8. Proposed approximate multiplier removes the insignificant partial products.

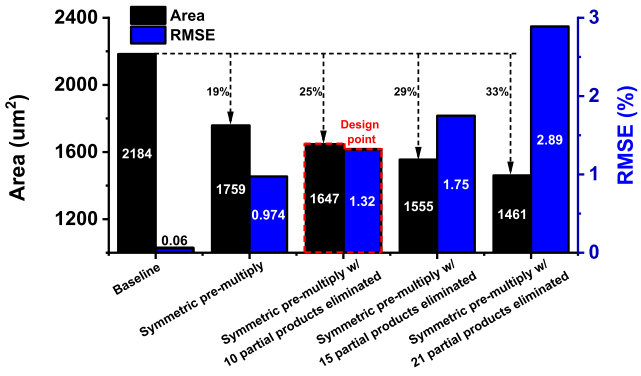


Fig. 9. Proposed approximate arithmetic techniques reduce the arithmetic hardware area by 25% at an RMSE of 1.32%. We use data that follows a Gaussian distribution for both weights and inputs.

unit based on zero_check[3:0] signals. We inserted multiplexers (muxes) in EXP for this data-gating feature. It also gates the data inputs to the multipliers and adders in the MTS unit if the data are zero or effectively zero. The data is effectively zero if the total shift amount is 15 bits or more because we must shift the input mantissae by 8 bits, which makes it 0. Note that we have found that the strictly zero condition dominates the gating activity. The effectively zero condition occurs when the difference in exponents (and hence the bit-shift amount) exceeds 15. We do not see many such effectively zero cases in our testbenches.

We evaluate the energy-efficiency improvements from the proposed gating techniques. We run a SPICE simulation of the CIM macro by enabling/disabling each gating feature. We use the input and weight data with 50% sparsity. Fig. 11 shows that the three techniques combined can reduce the power consumption by 51%. The area overhead is estimated at only 4%. The gating effectiveness is highly dependent on data sparsity. If the weight and input data exhibit 50% sparsity, the row-wise wordline gating achieves an activation ratio of approximately 50%. The column-wise clock gating exhibits a lower activation ratio of 8% because it requires that every pair in a four-row group contain at least one zero. Finally, the input data gating achieves an activation rate of 75%, because the 50%-sparsity input and weight can satisfy the gating condition 75% of the time.

E. DW Convolution Support

The conventional CIM macro has a long column of bitcells to support the accumulation of a large number of products, as the number of products is filter width × filter height × number of input channels. However, a DW convolution layer requires accumulating only a few products, as the accumulation length is filter width × filter height.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:21:40 UTC from IEEE Xplore. Restrictions apply.

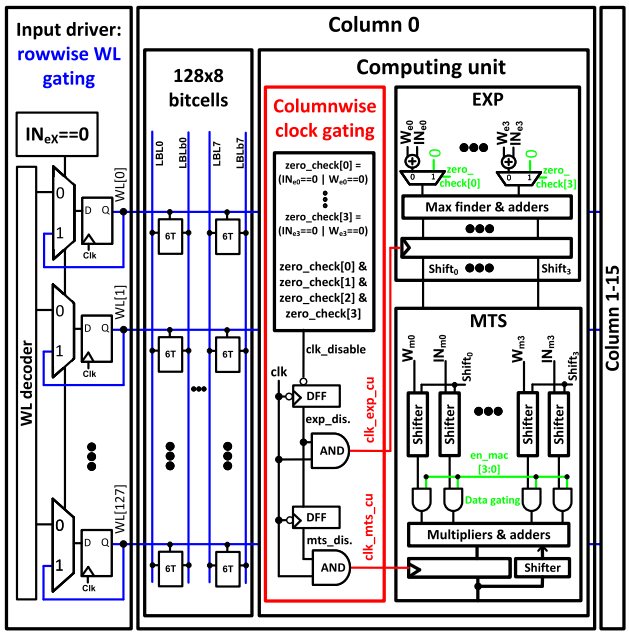


Fig. 10. Proposed row-wise WL-gating, column-wise clock-gating, and data-gating techniques.

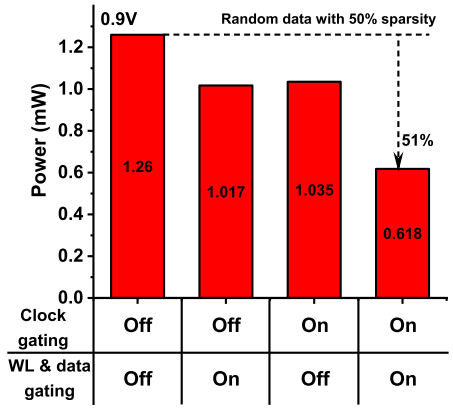


Fig. 11. Proposed gating techniques reduce power consumption by 51%. We use randomly generated inputs and weights that follow a Gaussian distribution and 50% sparsity.

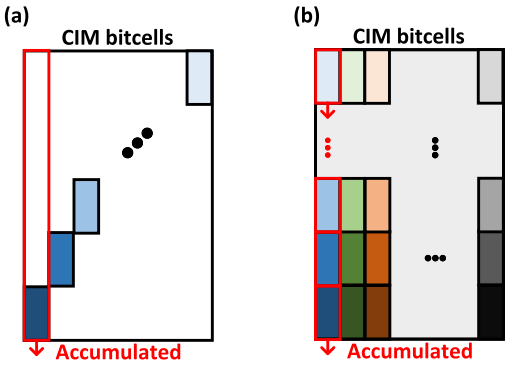


Fig. 12. (a) Conventional CIM macros suffer from low bitcell utilization for a DW layer, as they only use a small subset of bitcells in the diagonal region. It also performs unnecessary accumulation of zeros. (b) Proposed accumulator enhances utilization and reduces cycle counts by supporting flexible accumulation lengths.

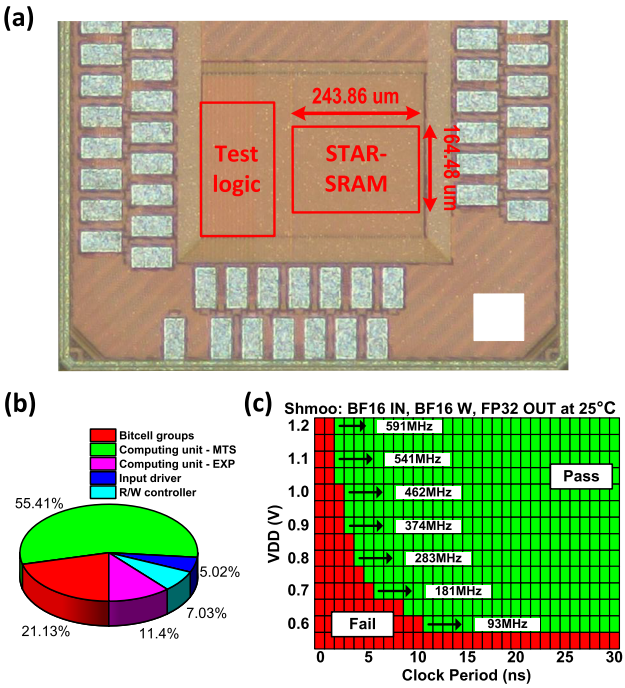


Fig. 13. STAR-SRAM measurement results. (a) Die micrograph. (b) Area breakdown. (c) Shmoo measurement.

This large difference in accumulation length can make the conventional CIM macro inefficient for mapping a DW layer. For example, a DW layer having a filter width of 3 and a filter height of three requires only nine bitcells in a column. The remaining bitcells in the column must store zeros. We then map the next filter in a different column. Fig. 12(a) describes such a low utilization of the conventional CIM macro if it maps a DW layer. Furthermore, the macro still needs to accumulate all products in the column, even when many are zeros, resulting in unnecessary delays and energy consumption.

In this work, we aim to update our macro to better support a DW convolution layer. To do so, we modify the accumulator (in the MTS module) to configure the accumulation length (the number of products to accumulate) from 4 to 64, with a granularity of 4. Specifically, we add a counter and a multiplexer so that accumulation stops when accum_length reaches DSCNN_count. Fig. 6 shows this modification.

The proposed configurable accumulator fully utilizes each column, enabling us to store more kernels from multiple channels within the available columns. However, those columns do not share the same inputs. To maximally utilize all columns, we could deploy a dedicated set of input drivers for each column. However, this would incur substantial area overhead–estimated at 50% of the macro area. Those additional drivers also degrade the macro's performance in the regular convolution mode, which we must avoid. Therefore, we decided to time-share a set of input drivers across the 16 columns of STAR-SRAM. In DW mode, it activates a single column at a time to process the input vector for a channel.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:21:40 UTC from IEEE Xplore. Restrictions apply.

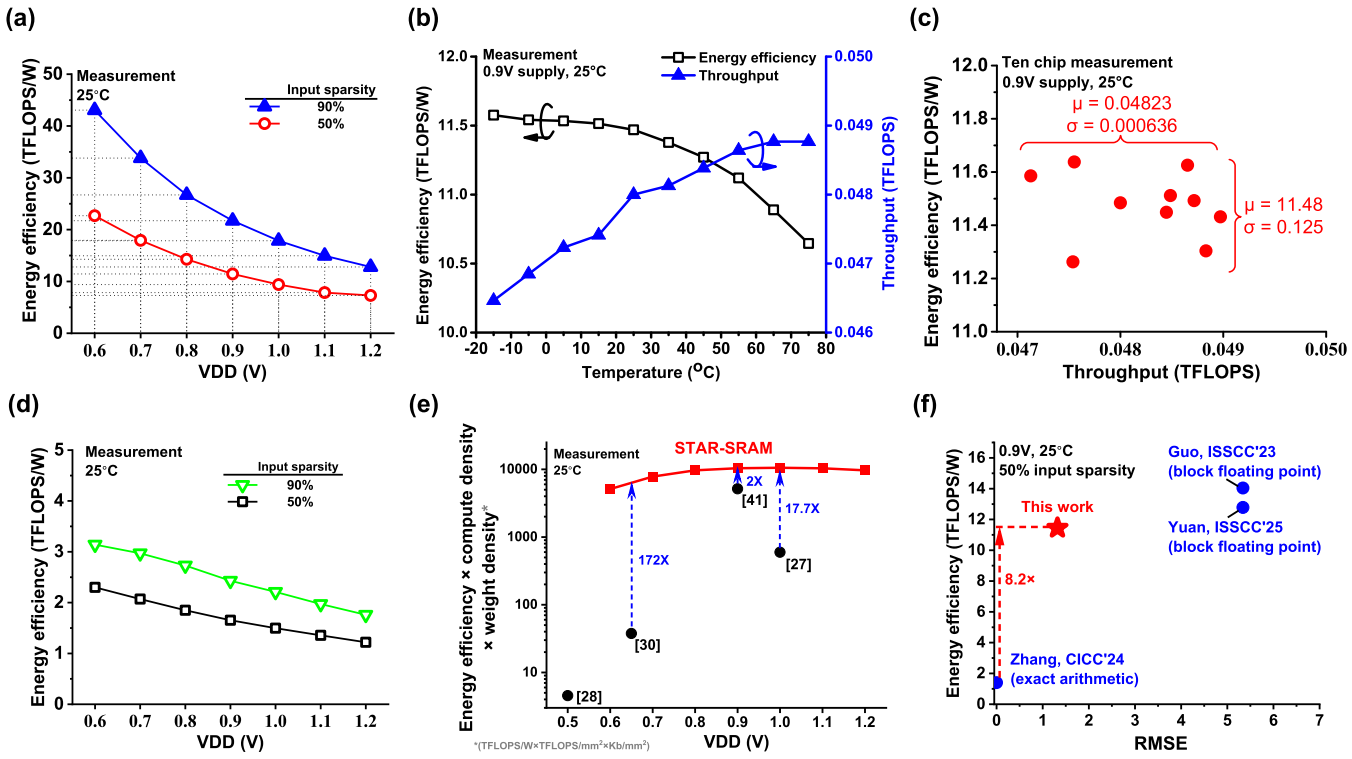


Fig. 14. STAR-SRAM measurement results. (a) Energy efficiency across VDDs for the standard convolution. (b) Energy efficiency and throughput across temperatures at 0.9 V. (c) Energy efficiency and throughput across ten chips at 0.9 V. (d) Energy efficiency across VDDs for DW convolutions. (e) ECWP across VDDs compared to prior work. Lee et al. [27] and Yue et al. [41] supported the same/similar precision of BF16/FP16. [28] supports variable precision from FP8 to FP32 and [30] supports FP32. (f) Energy efficiency and RMSE comparisons with prior work.

Indeed, time-sharing the input drivers across columns reduces computing throughput, but it saves silicon area and, more importantly, does not compromise area or energy efficiency for standard convolution computation.

We evaluate performance improvements in the DW mode using the typical DW layer of MobileNetv1, with an accumulation length of 9. The STAR-SRAM's longest accumulation length is 64. If it uses this length for the DW layer, it takes 16 cycles. However, if we set the accumulation length to 9 for the same layer, STAR-SRAM can complete the task in three cycles. This improvement marks a 5.3× increase in throughput and a 4× increase in hardware utilization. The area overhead is ~0.2% of the arithmetic hardware.

Last but not least, we would like to draw attention to the alternative approach [40], which proposes storing feature maps rather than weight data in a conventional CIM macro. This setup exhibits a longer accumulation length and supports a DW layer better than the conventional setup. It, however, requires some level of feature data replication, undermining effective data density and energy efficiency.

IV. Measurement Results and Comparisons

A. Measurement Results

We prototyped the STAR-SRAM test chip in 28-nm CMOS. Fig. 13(a) shows the chip photograph. The 16-Kbit macro occupies 0.049 mm^2, achieving a weight density of 400 Kb/mm^2. Fig. 13(b) shows the area breakdown. The arithmetic hardware takes 66.81% of the total area, while the bitcells take 21.13%. We characterize the macro across 0.6–1.2 V while it performs computing operations in the BF16 precision. Fig. 13(c) shows the shmoo plot, where the macro achieves the maximum clock frequency of 591 MHz at 1.2 V and 93 MHz at 0.6 V.

Fig. 14(a) shows the energy efficiency at the maximum clock frequency at each supply voltage. STAR-SRAM attains 12.79–43.06 TFLOPS/W at 90% input sparsity (90% zero-valued data) and 7.29–22.67 TFLOP/W at 50% input sparsity for standard convolution layers. We maintain sparsity in the weight data at 50%.

We also measure energy efficiency and throughput across different temperatures. As shown in Fig. 14(b), the energy efficiency decreases with higher temperatures, while throughput increases 5% from −15 °C to 75 °C at 0.9 V, mainly because of the increased drain current of transistors at high temperature. We also measured energy efficiency and throughput across ten chips at the nominal voltage of 0.9 V [Fig. 14(c)]. It gives a standard deviation-to-mean ratio (σ/μ) of 0.013 for throughput and 0.0109 for energy efficiency.

We also measure the macro's energy efficiency in the DW mode. We map the DW layers in MobileNetv1, where the macro exhibits the bitcell utilization ratio of 56%. As shown in Fig. 14(d), STAR-SRAM achieves 1.76–3.14 TFLOPS/W at 90% input sparsity and 1.22–2.3 TFLOPS/W at 50% input sparsity. The low efficiency compared to the regular convolution mode is attributed to low hardware utilization,

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:21:40 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISONS TO THE STATE-OF-THE-ART SRAM-BASED FLOATING-POINT CIM MACROS

	This work	JSSC25 [30]	ISSCC25 [41]	ISSCC25 [13]	VLSI21 [27]	ESSCIRC22 [28]	JSSC24 [29]	JSSC24 [31]	ISSCC24 [16]
Technology (nm)	28	28	28	28	28	28	22	28	28
MAC implementation	Digital	Digital	Digital	Digital	Digital	Digital	Analog + digital	Digital	Analog + digital
Supply voltages (V)	0.6-1.2	0.6-1.0	0.62-0.9	0.55-0.9	0.68-1.1	0.5-0.9	0.6-0.8	0.6-0.9	0.7-0.95
Operating frequency (MHz)	93-591	115-290	100-525	188.7	250	53-403	156	147	208
SRAM cell	6T	6T	10T	6T	6T	8T	6T push rule	6T	6T
Macro size	16Kb	32Kb	224Kb	32Kb	80Kb	16Kb	832Kb	64Kb	192Kb
Macro area (mm²)	0.04	0.109	1.6	0.043	0.201	0.0439	18*2	0.146	1.94
Weight density (Kb/mm²)	400	292.57	140	749.57	397.2	364.47	46.22*2	438.36	98.97
Input and weight precision	BF16	FP32	FP16	FP8, BF16	BF16	FP8-FP32	BF16	BF16	BF16
Output precision	FP32	FP32	FP32	FP8, BF16	FP32	FP8-FP32	FP32	BF16	FP32
Control for alignment	Pre-multiply alignment for IN and W based on (W_E+IN_E)	Pre-accumulation alignment	Pre-accumulation alignment	Block floating point	Pre-accumulation alignment	Pre-accumulation alignment	Pre-multiply alignment for IN based on (W_E+IN_E)	Block floating point	Block floating point
Compute density (TFLOPS/mm²)	1.89 (1.2V) 0.297 (0.6)	0.0257 (1.0V) 0.0155 (0.65V)	2.44 (0.9V)	2.24 (0.9V)	0.2054 (1.1V) 0.0329 (0.76V)	0.058 (0.9V, FP8) 0.0076 (0.5V, FP8)	0.071 (0.8V)*2	2.05 (0.9V)*4	2.21 (0.95V) 1.02 (0.7V)
Energy efficiency (TFLOPS/W)*6	12.79 (1.2V) 43.06 (0.6V)	8.3 (0.65V)	15.1 (0.9V)	12.78 (0.9V)	7.3 (1.1V) 13.7 (0.76V)	1.644 (0.5V, FP8)	62.14 (0.8V) 70.21 (0.6V)	14.04 (0.9V) 31.6 (0.6V)	36.41 (0.95V) 72.12 (0.7V)
ECWP = energy efficiency × compute density × weight density	10561 (1V) 5123 (0.6V)	38 (0.65V)	5158 (0.9V)	21458 (0.9V)	596 (1V)	4.55 (0.5V)	203.9 (0.8V)*2	12617 (0.9V)	7964 (0.95V)
RMSE (%)*1	1.32	0	0	5.34	0.08	0.72	6.13*4	5.34	5.34*4
Inference accuracy loss	0.13% (CIFAR100 w/ ResNet18)	N/R	N/R	0.10% (CIFAR100 w/ VGG-19)	N/R	N/R	0.07% (CIFAR100 w/ ResNet20)	0.17% (CIFAR100 w/ ResNet18)	0.05% (CIFAR100 w/ ResNet18)

*1: RMSE = $\sqrt{\frac{1}{n}\sum_{i=1}^{n}(\frac{\hat{y}_i-y_i}{\hat{y}_i})^2}$, simulated using inputs and weights following Gaussian distribution
*2: include test logic since macro area not reported
*3: N/R: not reported
*4: error due to analog PVT variations not reported
*5: one operation (OP) stands for one multiplication or one addition
*6: Energy efficiency is measured at 90% input sparsity

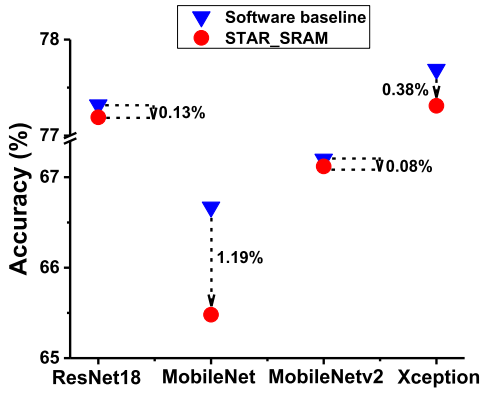


Fig. 15. STAR-SRAM accuracy results for CIFAR100.

as the STAR-SRAM shares a set of input drivers across columns.

We also estimate the macro's computational accuracy. We map four models: ResNet18, MobileNet, MobileNetv2, and Xception, all trained for the CIFAR100 dataset. Fig. 15 summarizes the results. For ResNet18, the macro achieves 77.36% inference accuracy, only 0.13% lower than the software baseline (BF16, no approximation). The other three models, which contain both regular and DW convolution layers, achieve 65.5%–77.3% accuracy, only 0.08%–1.19% lower than the software baseline.

B. Comparisons

Table I compares STAR-SRAM [19] with recent state-of-the-art floating-point CIM macros [13], [16], [27], [28], [29], [30], [31], [41]. We include only those prior works that support floating-point arithmetic and also report energy efficiency, weight density, compute density, and RMSE.

STAR-SRAM aims to strike a balance across all major metrics. It successfully achieves this goal, setting a state-of-the-art ECWP of 10 561 at 1 V and 5123 at 0.6 V. As summarized in Fig. 14(e), STAR-SRAM outperforms all the prior works in ECWP, across supply voltages from 0.6 to 1.2 V, by 2× to 172×. Regarding the individual metrics, STAR-SRAM achieves the third-best weight density after [13], [31]; fifth-best compute density after [13], [16], [31], [41]; and third-best energy efficiency after [16], [29], all at 1 V. Note that [27] and [30] reported both macro- and accelerator-level metrics. We use the macro-level metrics in our comparisons. [16] can support both inner and outer-product calculations.

We also compare STAR-SRAM with prior work using RMSE metrics. We compute the RMSE on the MAC results based on the inputs and weights following Gaussian distributions. STAR-SRAM exhibits an RMSE of 1.32%. Some prior works [13], [16], [31] adopt block floating-point arithmetic, thereby eliminating the need for hardware for the alignment operation. However, the block floating-point arithmetic increases the RMSE to > 5% because it pre-aligns all mantissas to the largest exponent in the block. Macros that adopt analog arithmetic hardware also exhibit a considerable amount of RMSE of > 5% even before including potential RMSE degradation across PVT variations [29]. Fig. 14(f) summarizes the tradeoff between RMSE and energy efficiency. STAR-SRAM achieves 8.2× higher energy efficiency than the exact arithmetic macro [42], while reducing RMSE by

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:21:40 UTC from IEEE Xplore. Restrictions apply.

4.1× as compared to the block floating point macro [13], [16], [31] at a similar energy efficiency. Note that we do not consider inference accuracy loss a critical metric for the comparison. This is because they all achieve small inference accuracy losses, ranging from 0.05% to 0.17%. Also, they use different model topologies (e.g., VGG-19, ResNet18, and ResNet20), different arithmetic precisions, different weight parameter values, and different precisions for non-convolution layers, which makes fair comparison difficult.

V. Conclusion

This article presents STAR-SRAM, a DCIM macro for BF16 arithmetic. We optimize key design parameters to simultaneously maximize energy efficiency, compute density, and weight density. We also develop the pre-multiply symmetric alignment and approximate multiplier schemes to reduce the arithmetic hardware area while maintaining low RMSE. Additionally, sparsity-aware wordline, clock, and data gating techniques further enhance energy efficiency. Fabricated in a 28-nm CMOS, STAR-SRAM achieves up to 43.06 TFLOPS/W, 1.89 TFLOPS/mm^2, and 400 Kb/mm^2. STAR-SRAM achieves state-of-the-art performance in the combination of the three metrics.

Appendix
Model Derivations

A. Models for Exact Arithmetic Macros

1) Weight Density: The weight density (d_w) of the CIM macro represents how many weight bits a macro stores per unit area. Ignoring the area of the column and row peripherals, we can formulate d_w as follows:

$$d_w \approx \frac{N_{6T}}{A_{column}} \approx \frac{N_{6T}}{k_{area} \cdot N_T} \tag{1}$$

where A_{column} is the area of a column, k_{area} is a hyperparameter, N_{6T} is the number of 6T SRAM bitcells in one column, and N_T is the number of transistors in N_{col} columns. We assume that A_{macro} is proportional to N_T.

We then develop the formula for N_T. We assume we use only the 6 T bitcell, as it is often the optimal choice for a hardware reuse architecture [17], [26]. Therefore, the bitcells in a column contribute a total of $6{\cdot}N_{6T}$ transistors. On the other hand, we define the number of transistors of one computing unit (in a column) as $N_{Tcompute}$. Since the macro has N_{col} columns, we can formulate N_T as

$$N_T = \left(6 \cdot N_{6T} + N_{T_{compute}}\right) \cdot N_{col}. \tag{2}$$

We then develop the formula for $N_{Tcompute}$. Again, since we assume the macro uses exact arithmetic, we name it $N_{Tcompute,exact}$. We can formulate it as the sum of the transistors of the three building blocks as follows:

$$N_{T_{compute,exact}} = N_{T_{EXP,exact}} + N_{T_{MTS,exact}} + N_{T_{ACC,exact}} \tag{3}$$

where $N_{T_{EXP,exact}}$, $N_{T_{MTS,exact}}$, and $N_{T_{ACC,exact}}$ are the number of transistors in the EXP, MTS, and ACC units, respectively. Now, we start to formulate $N_{T_{EXP,exact}}$. We assume that the EXP unit processes N_{input} elements of the input vector and N_{input} elements of the weight vector in each cycle. Therefore, it requires N_{input} 8b ripple carry adders (RCAs) for shift generation. Therefore, we can formulate it as

$$N_{T_{EXP,exact}} = N_{input} \cdot N_{T_{RCA,8b}} = N_{input} \cdot 8 \cdot N_{T_{FA}} \tag{4}$$

where $N_{T_{RCA,8b}}$ is the number of transistors for an 8b RCA and $N_{T_{FA}}$ is the number of transistors in a full adder cell, the latter ranges from 12 [18] to 28 [43].

On the other hand, the MTS unit has N_{input} 2-input 8b multipliers and an adder tree with N_{input} inputs. The adder tree consists of $\log_2 N_{input}$ stages in total. The first stage consists of $N_{input}/2$ additions. Every addition between two numbers requires an 8-bit comparator and an 8-bit RCA to generate the shift, and a 1–18-bit shifter and an 18-bit adder for addition. Higher stages are similar, but with more bit precision and fewer additions. Therefore, we can formulate it as

$$N_{T_{MTS,exact}} = N_{input} \cdot N_{T_{mul,8b}} + \Sigma_{i=0}^{log_2 N_{input}-1} \left\{2^i \cdot \left(N_{T_{compare,8b}} + N_{T_{RCA,8b}} + N_{T_{shift,BW-i}} + N_{T_{RCA,BW-i}}\right)\right\} \tag{5}$$

where $N_{T_{mul,8b}}$ is the number of transistors of a two-input 8b multiplier, $N_{T_{shift,ib}}$ is the number of transistors of a 1-ib shifter, and $N_{T_{compare,8b}}$ is the number of transistors in an 8b comparator. BW represents the bitwidth. For bit-parallel, BW can be formulated as

$$BW = 18 + log_2\left(N_{input}\right) - 1. \tag{6}$$

The BW after the two-input 8b multiplier starts at 18b, since we pad the 16b multiplication result with one sign bit and one implicit leading MSB bit (1.mantissa[7:0]).

On the other hand, for bit-serial input, we can formulate BW as

$$BW = 10 + log_2\left(N_{input}\right) - 1. \tag{7}$$

The BW after the two-input 8b multiplier is 10b, since we pad the 8b multiplication result with a sign bit and an implicit leading MSB bit.

Finally, the ACC unit has a 2-input 23-bit adder, a 1-23-bit shifter, and 23 D flip-flops (DFFs). We formulate $N_{T_{ACC}}$ as the following:

$$N_{T_{ACC}} = N_{T_{RCA,23b}} + N_{T_{shift,23b}} + N_{T_{DFF,23b}} \tag{8}$$

where $N_{T_{RCA,23b}}$ is the number of transistors of a 23b RCA and $N_{T_{DFF,Xb}}$ is the number of transistors of an Xb DFF.

By combining (1) and (2), we can now complete the d_w model as follows:

$$d_w \approx \frac{N_{6T}}{k_{area} \cdot \left(6 \cdot N_{6T} + N_{T_{compute}}\right) \cdot N_{col}} \tag{9}$$

where $N_{Tcompute}$ can be obtained by (3).

2) Energy Efficiency: We now derive the model for energy efficiency, e_e, defined as the number of operations performed per unit of energy consumption. Therefore, we can formulate e_e as

$$e_e = \frac{N_{OP}}{E} = \frac{N_{OP}}{N_{cycle} \cdot P \cdot t_{clk}} \tag{10}$$

where N_{cycle} is the number of cycles to complete those MACs, E and P are the macro energy and power consumption to

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:21:40 UTC from IEEE Xplore. Restrictions apply.

complete those MACs, and t_{clk} is the operating clock period of the macro. N_{OP} is the number of MACs that a macro performs, which can be formulated as follows:

$$N_{OP} = 2 \cdot N_{col} \cdot N_{row}. \tag{11}$$

We can further develop the nominator of (10) as follows:

$$\begin{aligned} N_{cycle} \cdot P \cdot t_{clk} &= N_{cycle} \cdot \alpha \cdot C \cdot {V_{DD}}^2 \cdot (1/t_{clk}) \cdot t_{clk} \\ &= N_{cycle} \cdot \alpha \cdot C \cdot {V_{DD}}^2 \end{aligned} \tag{12}$$

where α is the nodal switching activity of a macro (between 0 and 1), V_{DD} is the operating voltage, and C is the switching capacitance of a macro. We opted to use a single nodal switching activity (α) for simpler model development, whereas the macro has several modules with distinct switching activities. We pessimistically set the nodal switching activity to 0.5 for the 50% sparsity data. We can further simplify (12) into

$$N_{cycle} \cdot \alpha \cdot C \cdot {V_{DD}}^2 \approx N_{cycle} \cdot \alpha \cdot k_{e_e} \cdot N_T \tag{13}$$

where we approximate $C \cdot {V_{DD}}^2$ to $k_{e_e} \cdot N_T$ since C is roughly proportional to N_T.

Finally, by combining (10), (11) and (13), we can formulate e_e to

$$e_e \approx \frac{2 \cdot N_{col} \cdot N_{row}}{N_{cycle} \cdot \alpha \cdot k_{e_e} \cdot N_T}. \tag{14}$$

3) Compute Density: Now we switch gears to the development of the compute density model (d_c), which indicates the computation throughput of a macro per unit area. We can start to formulate it as follows:

$$\begin{aligned} d_c &= \frac{N_{OP} / \left(N_{cycle} \cdot t_{clk}\right)}{A_{macro}} \\ &\approx \frac{N_{OP} / \left(N_{cycle} \cdot t_{clk}\right)}{k_{area} \cdot N_T} \end{aligned} \tag{15}$$

where we assume the area to be proportional to the number of transistors.

In (15), we derive all the variables except t_{clk}. Therefore, we now derive the t_{clk} formula by approximating it as the critical path delay of the computing unit, $t_{comp_{exact}}$, which can be formulated as

$$t_{comp_{exact}} = t_{MTS} + t_{ACC} \tag{16}$$

where t_{MTS} is the critical path delay of the MTS unit and t_{ACC} is the critical path delay of the ACC unit. We ignore the critical path delay of the EXP unit since it operates in parallel with the MTS unit.

For input bit-parallel, the MTS unit performs two-input 8b multiplications and an N_{input}-input adder tree operation. The adder tree consists of $\log_2 N_{input}$ stages, and each stage performs a shift generation (a 2-input comparison and a 2-input addition), a bit-shift operation, and a 2-input addition. Therefore, we can formulate t_{MTS} as

$$\begin{aligned} t_{MTS} &= t_{mul,8b} + \Sigma_{i=0}^{log_2 N_{input}-1} \\ &\left\{\left(t_{compare,8b} + t_{RCA,8b} + t_{shift,BW-i} + t_{RCA,BW-i}\right)\right\} \end{aligned} \tag{17}$$

where $t_{RCA,\ Xb}$ is the critical path delay for a Xb RCA, $t_{mul,8b}$ is the critical path delay of a two-input 8b multiplier, $t_{shift,ib}$ is the critical path delay of a 1-*i*b shifter, and $t_{compare,8b}$ is the critical path delay of a 8b comparator. For input bit-serial, $t_{mul,8b}$ is replaced with $t_{mul,8b1b}$ since the MTS unit performs 8b and 1b multiplications instead of two-input 8b multiplications.

Furthermore, we can formulate $t_{RCA,\ Xb}$ into

$$t_{RCA,Xb} = (X-1) \cdot t_c + t_s \tag{18}$$

where t_c is a carry-out delay of a FA cell and t_s is a sum-out delay. Also, we can formulate $t_{mul,8b}$ using array multiplier as

$$t_{mul,8b} = 13\ t_c + 7\ t_s + t_{and} \tag{19}$$

where t_{and} is the delay of an AND gate.

The ACC unit mainly performs a 2-input addition. Therefore, we can formulate t_{ACC} to

$$t_{ACC} = t_{RCA,23b}. \tag{20}$$

Finally, by combining (11), (15), (16), (17), and (20), we can formulate d_c to

$$d_c \approx \frac{2 \cdot N_{col} \cdot N_{row} / \left(N_{cycle} \cdot t_{clk}\right)}{k_{area} \cdot \left(6 \cdot N_{6T} + N_{T_{compute}}\right) \cdot N_{col}} \tag{21}$$

where t_{clk} can be approximated using (16) and $N_{Tcompute}$ can be obtained by (3).

4) ECWP: Finally, using (1), (14), and (21), we can formulate the product of energy efficiency, compute density, and weight density (ECWP) as

$$ECWP \approx \frac{N_{6T} \cdot N_{OP}^2}{k \cdot \alpha \cdot N_T^3 \cdot N_{cycle}^2 \cdot t_{clk}} \tag{22}$$

where k is $k_{d_w} \cdot k_{d_c} \cdot k_{e_e}$, t_{clk} can be approximated using (16) and N_T can be obtained by (2).

B. Additional Models for Approximate Arithmetic Macros

1) Number of Transistors: First, we derive the equation of the number of transistors in the computing unit for the macro having the pre-multiply alignment, which can be formulated as follows:

$$N_{T_{compute,premul}} = N_{T_{EXP,premul}} + N_{T_{MTS,premul}} + N_{T_{ACC}} \tag{23}$$

where $N_{T_{EXP,premul}}$ and $N_{T_{MTS,premul}}$ are the number of transistors of the EXP and MTS units, respectively. $N_{T_{ACC}}$ remains the same as in the exact arithmetic case, as the pre-multiply alignment technique does not make any modification to it.

The EXP unit in pre-multiply alignment involves operations to find the largest exponent among the N_{input} data and generate N_{input} shift values. Therefore, we can formulate $N_{T_{EXP,premul}}$ as the following:

$$\begin{aligned} N_{T_{EXP,premul}} &= N_{input} \cdot 2 \cdot N_{T_{RCA,8b}} \\ &+ \Sigma_{i=0}^{log_2 N_{input}-1} \left\{2^i \cdot N_{T_{compare,8b}}\right)\}. \end{aligned} \tag{24}$$

On the other hand, the MTS unit shifts the input and weight mantissae and performs MAC operations. We can formulate $N_{T_{MTS,premul}}$ as follows:

$$\begin{aligned} N_{T_{MTS,premul}} &= N_{input} \cdot \left(2N_{T_{shift,8b}} + N_{T_{mul,8b}}\right) \\ &+ \Sigma_{i=0}^{log_2 N_{input}-1} \left\{2^i \cdot \left(N_{T_{RCA,BW-i}}\right)\right\}. \end{aligned} \tag{25}$$

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:21:40 UTC from IEEE Xplore. Restrictions apply.

Note that the pre-multiply alignment removes the shift-generation hardware from the MTS unit but adds a simpler version to the EXP unit. Fig. 4(b) shows that the pre-multiply alignment can reduce the macro area by 27%.

We also formulate the number of transistors of the computing unit for the macro using block floating point $N_{T_{compute,bfp}}$, which can be formulated as follows:

$$N_{T_{compute,bfp}} = N_{T_{EXP,bfp}} + N_{T_{MTS,bfp}} + N_{T_{ACC}} \tag{26}$$

where $N_{T_{EXP,bfp}}$ and $N_{T_{MTS,bfp}}$ are the number of transistors of the EXP and MTS units, respectively. We formulate $N_{T_{EXP,bfp}}$ as the following:

$$N_{T_{EXP,bfp}} = N_{input} \cdot N_{T_{RCA,8b}} + \Sigma_{i=0}^{log_2 N_{input}-1} \left\{2^i \cdot N_{T_{compare,8b}}\right)\right\}. \tag{27}$$

We also formulate $N_{T_{MTS,premul}}$ as

$$N_{T_{MTS,bfp}} = N_{input} \cdot \left(N_{T_{shift,8b}} + N_{T_{mul,8b}}\right) + \Sigma_{i=0}^{log_2 N_{input}-1} \left\{2^i \cdot \left(N_{T_{RCA,BW-i}}\right)\right\}. \tag{28}$$

The block floating-point technique is similar to pre-multiply alignment in the input exponent and mantissa. But it eliminates the logic to sum the input and weight exponents in EXP and also avoids the shifters for weight mantissae in MTS. This is because block floating point shares the same weight exponent. Fig. 4(b) shows that the block floating point technique can reduce the area of the macro by 34% as compared to the exact arithmetic hardware.

2) RMSE: The approximate arithmetic produces errors, which demand the development of the RMSE model. Making a closed-form model for RMSE is non-trivial. Therefore, we create a functional (software) model of the CIM's vector–matrix multiplication and estimate RMSE by feeding it randomly generated inputs.

We use the following formula for the RMSE calculation:

$$RMSE = \sqrt{\frac{\sum_{i=1}^{N}\left(\frac{\hat{y}_i - y_i}{y_i}\right)^2}{N}} \tag{29}$$

where we assume the multiplication between an N × N weight matrix and an N-dimensional input vector. We generate the matrix and vector based on a Gaussian distribution. y is the output vector of the exact arithmetic, while $\hat{y}$ is the counterpart of the approximate arithmetic.

3) Critical Path Delay: Also, we formulate the t_{comp} adopting pre-multiply alignment ($t_{comp_{premul}}$) as the following:

$$t_{comp_{premul}} = 2\ t_{RCA,8b} + t_{shift,8b} + t_{mul,8b} + \Sigma_{i=0}^{log_2 N_{input}-1} \left\{\left(t_{RCA,BW-i} + t_{compare,8b}\right)\right\} + t_{ACC}. \tag{30}$$

And we can formulate t_{comp} using block floating point ($t_{comp_{bfp}}$) as the following:

$$t_{comp_{bfp}} = t_{RCA,8b} + t_{shift,8b} + t_{mul,8b} + \Sigma_{i=0}^{log_2 N_{input}-1} \left\{\left(t_{RCA,BW-i} + t_{compare,8b}\right)\right\} + t_{ACC}. \tag{31}$$

REFERENCES

[1] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–11.

[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Hum. Lang. Technol.*, vol. 1. Minneapolis, MI, USA: Association for Computational Linguistics, Jun. 2019, pp. 4171–4186.

[3] T. B. Brown et al., "Language models are few-shot learners," in *Proc. NIPS*, 2020, pp. 1877–1901.

[4] A. Dosovitskiy et al., "An image is worth 16×16 words: Transformers for image recognition at scale," 2020, *arXiv:2010.11929*.

[5] H. Touvron et al., "LLaMAA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.

[6] N. Verma et al., "In-memory computing: Advances and prospects," *IEEE Solid State Circuits Mag.*, vol. 11, no. 3, pp. 43–55, Summer. 2019.

[7] S. Yin, Z. Jiang, J.-S. Seo, and M. Seok, "XNOR-SRAM: In-memory computing SRAM macro for binary/ternary deep neural networks," *IEEE J. Solid-State Circuits*, vol. 55, no. 6, pp. 1733–1743, Jun. 2020.

[8] Z. Jiang, S. Yin, J.-S. Seo, and M. Seok, "C3SRAM: An in-memory-computing SRAM macro based on robust capacitive coupling computing mechanism," *IEEE J. Solid-State Circuits*, vol. 55, no. 7, pp. 1888–1897, Jul. 2020.

[9] B. Zhang et al., "MACC-SRAM: A multistep accumulation capacitor-coupling in-memory computing SRAM macro for deep convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 59, no. 6, pp. 1938–1949, Jun. 2024.

[10] R. Wan, C.-T. Lin, B. Zhang, Y. Tsividis, and M. Seok, "AACIM: A 2785-TOPS/W, 161-TOP/mm^2, <1.17%-RMSE, analog-in analog-out computing-in-memory macro in 28 nm," *IEEE J. Solid-State Circuits*, early access, Oct. 28, 2025, doi: 10.1109/JSSC.2025.3622309.

[11] P.-C. Wu et al., "An integer-floating-point dual-mode gain-cell computing-in-memory macro for advanced AI edge chips," *IEEE J. Solid-State Circuits*, vol. 60, no. 1, pp. 158–170, Jan. 2025.

[12] X. Wang et al., "14.3 a 28 nm 17.83-to-62.84TFLOPS/W broadcast-alignment floating-point CIM macro with non-two's-complement MAC for CNNs and transformers," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2025, pp. 254–256.

[13] Y. Yuan et al., "14.5 a 28 nm 192.3TFLOPS/W accurate/approximate dual-mode-transpose digital 6T-SRAM CIM macro for floating-point edge training and inference," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2025, pp. 258–260.

[14] C.-Y. Chang, C.-T. Huang, Y.-C. Chuang, K.-C. Chou, and A.-Y. Wu, "BFP-CIM: Runtime energy-accuracy scalable computing-in-memory-based DNN accelerator using dynamic block-floating-point arithmetic," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 71, no. 5, pp. 2079–2092, May 2024.

[15] W. Jiang, C. Caron, P. Avasare, M. Pauwels, M. Verhelst, and W. Dehaene, "HUNBN, a 1.77MB digital in-memory-compute SoC for edge applications achieving 126 TOPs/W (4b) at macro level and 24 TOPs/W at SoC level," in *Proc. IEEE Eur. Solid-State Electron. Res. Conf. (ESSERC)*, Sep. 2024, pp. 137–140.

[16] Y. Yuan et al., "34.6 A 28 nm 72.12TFLOPS/W hybrid-domain outer-product based floating-point SRAM computing-in-memory macro with logarithm bit-width residual ADC," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2024, pp. 576–578.

[17] J. Oh, C.-T. Lin, and M. Seok, "D6CIM: 60.4-TOPS/W, 1.46-TOPS/mm^2, 1005-Kb/mm^2 digital 6T-SRAM-based compute-in-memory macro supporting 1-to-8b fixed-point arithmetic in 28-nm CMOS," in *Proc. ESSCIRC IEEE 49th Eur. Solid State Circuits Conf. (ESSCIRC)*, Sep. 2023, pp. 413–416.

[18] C.-T. Lin et al., "DIMCA: An area-efficient digital in-memory computing macro featuring approximate arithmetic hardware in 28 nm," *IEEE J. Solid-State Circuits*, vol. 59, no. 3, pp. 960–971, Mar. 2024.

[19] C.-T. Lin, J. Oh, K. Lee, and M. Seok, "STAR-SRAM: 43.06-TFLOPS/W, 1.89-TFLOPS/mm2, 400-Kb/mm^2 floating-point SRAM-based digital computing-in-memory macro in 28-nm CMOS," in *Proc. IEEE Custom Integr. Circuits Conf. (CICC)*, Apr. 2024, pp. 1–2.

[20] M. Ashok, S. Maji, X. Zhang, J. Cohn, and A. P. Chandrakasan, "Digital in-memory compute for machine learning applications with input and model security," *IEEE J. Solid-State Circuits*, vol. 60, no. 9, pp. 3390–3402, Sep. 2025.

[21] J. Yang, T. Li, W. Romaszkan, P. Gupta, and S. Pamarti, "A 65-nm digital stochastic compute-in-memory CNN processor with 8-bit precision," *IEEE J. Solid-State Circuits*, vol. 60, no. 10, pp. 3749–3761, Oct. 2025.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:21:40 UTC from IEEE Xplore. Restrictions apply.

[22] H. Diao et al., "A computing-in-memory engine supporting one-shot floating-point NN inference and on-device fine-tuning for edge AI," *IEEE J. Solid-State Circuits*, vol. 60, no. 9, pp. 3403–3415, Sep. 2025.
[23] H. Wu et al., "A 28-nm 88.3-TFLOPS/W POSIT-approximate-calculation-based digital computing-in-memory macro incorporating final-cycle fusion and joint skipping," *IEEE J. Solid-State Circuits*, vol. 61, no. 4, pp. 1–15, Apr. 2026.
[24] C.-T. Lin, P. X. Huang, J. Oh, D. Wang, and M. Seok, "IMCU: A 28-nm digital in-memory computing-based microcontroller unit for TinyML," *IEEE J. Solid-State Circuits*, vol. 59, no. 8, pp. 2684–2693, Aug. 2024.
[25] B. Zhang et al., "PIMCA: A programmable in-memory computing accelerator for energy-efficient DNN inference," *IEEE J. Solid-State Circuits*, vol. 58, no. 5, pp. 1436–1449, May 2023.
[26] J. Oh, C.-T. Lin, and M. Seok, "D6CIM: 60.4-TOPS/W all-digital 6T-SRAM-based compute-in-memory macro supporting 1-to-8b fixed-point arithmetic in a 28-nm CMOS," *IEEE Trans. Circuits Syst. I, Reg. Papers*, early access, Feb. 21, 2026, doi: 10.1109/TCSI.2026.3665839.
[27] J. Lee et al., "A 13.7 TFLOPS/W floating-point DNN processor using heterogeneous computing architecture with exponent-computing-in-memory," in *Proc. Symp. VLSI Circuits*, Jun. 2021, pp. 1–2.
[28] S. Jeong, J. Park, and D. Jeon, "A 28nm 1.644TFLOPS/W floating-point computation SRAM macro with variable precision for deep neural network inference and training," in *Proc. IEEE 48th Eur. Solid State Circuits Conf. (ESSCIRC)*, Sep. 2022, pp. 145–148.
[29] P.-C. Wu et al., "A floating-point 6T SRAM in-memory-compute macro using hybrid-domain structure for advanced AI edge chips," *IEEE J. Solid-State Circuits*, vol. 59, no. 1, pp. 196–207, Jan. 2024.
[30] Y. Wang et al., "TensorCIM: Digital computing-in-memory tensor processor with multichip-module-based architecture for beyond-NN acceleration," *IEEE J. Solid-State Circuits*, vol. 60, no. 2, pp. 734–747, Feb. 2025.
[31] A. Guo et al., "A 28-nm 64-kb 31.6-TFLOPS/W digital-domain floating-point-computing-unit and double-bit 6T-SRAM computing-in-memory macro for floating-point CNNs," *IEEE J. Solid-State Circuits*, vol. 59, no. 9, pp. 3032–3044, Sep. 2024.
[32] H. Zhang et al., "EasyACIM: An end-to-end automated analog CIM with synthesizable architecture and agile design space exploration," in *Proc. 61st ACM/IEEE Design Autom. Conf.*, Jun. 2024, pp. 1–6.
[33] J. Sun, P. Houshmand, and M. Verhelst, "Analog or digital in-memory computing? Benchmarking through quantitative modeling," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Oct. 2023, pp. 1–9.
[34] S. Spetalnick and A. Raychowdhury, "A practical design-space analysis of compute-in-memory with SRAM," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 4, pp. 1466–1479, Apr. 2022.
[35] J. Wang et al., "A 28-nm compute SRAM with bit-serial logic/arithmetic operations for programmable in-memory vector computing," *IEEE J. Solid-State Circuits*, vol. 55, no. 1, pp. 76–86, Jan. 2020.
[36] R. Sehgal, T. Thareja, S. Xie, C. Ni, and J. P. Kulkarni, "A bit-serial, compute-in-SRAM design featuring hybrid-integrating ADCs and input dependent binary scaled precharge eliminating DACs for energy-efficient DNN inference," *IEEE J. Solid-State Circuits*, vol. 58, no. 7, pp. 2109–2124, Jul. 2023.
[37] H. Kim, T. Yoo, T. T.-H. Kim, and B. Kim, "Colonnade: A reconfigurable SRAM-based digital bit-serial compute-in-memory macro for processing neural networks," *IEEE J. Solid-State Circuits*, vol. 56, no. 7, pp. 2221–2233, Jul. 2021.
[38] P. C. Knag et al., "A 617-TOPS/W all-digital binary neural network accelerator in 10-nm FinFET CMOS," *IEEE J. Solid-State Circuits*, vol. 56, no. 4, pp. 1082–1092, Apr. 2021.
[39] T. Njolstad and E. J. Aas, "Power consumption and performance of low-voltage bit-serial adders," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 1996, pp. 45–48.
[40] B. Wang et al., "A 28nm horizontal-weight-shift and vertical-feature-shift-based separate-WL 6T-SRAM computation-in-memory unit-macro for edge depthwise neural-networks," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 134–136.
[41] Z. Yue et al., "14.4 a 51.6TFLOPs/W full-datapath CIM macro approaching sparsity bound and <2–30 loss for compound AI," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2025, pp. 1–3.
[42] B. Zhang, S. Moon, and M. Seok, "A 1-TFLOPS/W, 28-nm deep neural network accelerator featuring online compression and decompression and BF16 digital in-memory-computing hardware," in *Proc. IEEE Custom Integr. Circuits Conf. (CICC)*, Apr. 2024, pp. 1–2.
[43] M. Alioto and G. Palumbo, "Analysis and comparison on full adder block in submicron technology," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 10, no. 6, pp. 806–823, Dec. 2002.



Chuan-Tung Lin (Student Member, IEEE) received the B.S. degree in electronics engineering from National Chiao Tung University, Hsinchu, Taiwan, in 2019, and the M.S. degree in electrical engineering from Columbia University, New York, NY, USA, in 2021, where he is currently pursuing the Ph.D. degree.

His research interests include energy-efficient architecture for machine learning algorithms and VLSI circuit and system design.



Jonghyun Oh (Member, IEEE) received the B.S. and Ph.D. degrees in electrical and computer engineering from Seoul National University, Seoul, South Korea, in 2015 and 2021, respectively.

In 2021, he was a Post-Doctoral Researcher with the Inter-University Semiconductor Research Center, Seoul National University. Since September 2021, he has been a Post-Doctoral Research Scientist with Columbia University, New York, NY, USA. His research interests include high-speed interfaces, integrated voltage regulators, synthesizable digital architectures, neuromorphic hardware accelerator design, and digital-type computing-in-memory macro design.

Dr. Oh has served as a reviewer for various journals, including IEEE JOURNAL OF SOLID-STATE CIRCUITS, IEEE SOLID-STATE CIRCUITS LETTERS, IEEE ACCESS, and IEEE OPEN JOURNAL OF CIRCUITS AND SYSTEMS.



Mingoo Seok (Senior Member, IEEE) received the B.S. degree (summa cum laude) in electrical engineering from Seoul National University, Seoul, South Korea, in 2005, and the M.S. and Ph.D. degrees in electrical engineering from the University of Michigan, Ann Arbor, MI, USA, in 2007 and 2011.

He was a member of the Technical Staff with Texas Instruments Inc., Dallas, TX, USA, in 2011. Since 2012, he has been with Columbia University, New York City, NY, USA, where he is currently an Associate Professor of electrical engineering. His current research interests include ultralow-power SoC design for emerging intelligent systems, machine-learning VLSI architecture and circuits, variation, voltage, and aging; thermal-adaptive circuits and architecture; on-chip integrated power circuits; and non-conventional hardware design, including in-memory computing and analog-mixed-signal computing hardware.

Prof. Seok serves as a Technical Program Committee Member for several conferences, including IEEE ISSCC and ACM/IEEE DAC. He received the 1999 Distinguished Undergraduate Scholarship from Korea Foundation for Advanced Studies, the 2005 Doctoral Fellowship from Korea Foundation for Advanced Studies, and the 2008 Rackham Pre-Doctoral Fellowship from the University of Michigan. He also received the 2009 Advanced Micro Devices (AMD)/Custom Integrated Circuits Conference (CICC) Scholarship Award for work on a picowatt voltage reference and the 2009 ACM/IEEE Design Automation Conference (DAC)/IEEE International Solid-State Circuits Conference (ISSCC) Design Contest for the 35-pW sensor platform design. He also received the 2015 National Science Foundation (NSF) Career Award and the 2019 Qualcomm Faculty Award. He also received the Best Paper Award from IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION (VLSI) SYSTEMS in 2022. He serves/served as an Associate Editor for IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS I: REGULAR PAPERS from 2013 to 2015, IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION (VLSI) SYSTEMS since 2015, and IEEE SOLID-STATE CIRCUITS LETTERS since 2017. He also served as a Guest Editor for IEEE JOURNAL OF SOLID-STATE CIRCUITS (JSSC).

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:21:40 UTC from IEEE Xplore. Restrictions apply.